

# CS 101 - Algorithms & Programming I

Spring 2026 - Lab 6

Due: Week of April 6, 2026

*Remember the **honor code** for your programming assignments.*

*For all labs, your solutions must conform to the CS101 style **guidelines**!*

*All data and results should be stored in variables (or constants where appropriate) with meaningful names.*

The objective of this lab is to learn static methods. Remember that analyzing the problems and designing the solutions on a piece of paper *before* starting implementation/coding is always a best practice.

For the methods below, you should **not** use any built-in methods that perform the mentioned tasks directly unless explicitly stated otherwise. The mentioned methods should be implemented from scratch.

## 0. Setup Workspace

Start VSC and open the previously created folder named `labs`. Now, under the `labs` folder, create a new folder named `lab6`. In this lab, you are to have 3 Java classes/files (under `labs/lab6` folder) as described below. A third Java file containing the revision should go under this folder as well. We expect you to submit a total of 4 files, including the revision, **without compressing** them. Do *not* upload other/previous lab solutions in your submission. The user inputs in the sample runs are shown in [blue](#).

First, we will design and implement some *static* methods. These methods will then be used in a program to determine the associated properties of the integers input by the user.

## 1. Drone Topography

Create a new/empty file of your own under the `lab6` folder named `Lab06_01.java` with a class with the same name.

The program will simulate a drone scanning the elevation of a terrain. First, the program should ask the user to enter the number of rows (`r`) and the number of columns (`c`) of the scanned grid. The terrain will be represented using a two-dimensional array of integers. After defining the dimensions, the user will enter `r*c` integers representing the elevation at each coordinate.

Your program must process this 2D array to identify all "Local Peaks." A cell is considered a local peak if its elevation is strictly greater than all of its valid, immediate neighboring cells (up, down, left, and right—do not check diagonals).

**Requirements:**

- You must implement a static method `boolean isPeak(int[][] grid, int row, int col)` that returns `true` if the specified cell is a peak, and `false` otherwise.
- Your method must carefully check the boundaries. Cells on the edges or corners of the grid will have fewer neighbors to compare against. You must handle this dynamically without causing an `ArrayIndexOutOfBoundsException`.
- The main method should print the 2D array in a grid format, followed by a list of all peaks found, displaying their coordinates (`row, col`) and elevation values. If no peaks are found, print "No peaks detected."

The following table:

#### Example 1:

```
Enter number of rows: 3
Enter number of columns: 3
Enter elevations:
1 2 1
2 5 2
1 2 1
Terrain Map:
1 2 1
2 5 2
1 2 1
Peaks found:
Peak at (1, 1) with elevation 5
```

#### Example 2:

```
Enter number of rows: 2
Enter number of columns: 4
Enter elevations:
9 2 4 8
3 1 6 7
Terrain Map:
9 2 4 8
3 1 6 7
Peaks found:
Peak at (0, 0) with elevation 9
Peak at (0, 3) with elevation 8
```

#### Example 3:

```
Enter number of rows: 3
Enter number of columns: 3
Enter elevations:
5 5 5
5 5 5
5 5 5
Terrain Map:
5 5 5
5 5 5
5 5 5
```

```
Peaks found:  
No peaks detected.
```

## 2. Dynamic Playlist Manager

Create a new/empty file of your own under the `lab6` folder named `Lab06_02.java` with a class with the same name.

In this part, you will design and implement a console-based Music Playlist System in Java that begins with fixed-size arrays and then transitions to dynamic `ArrayList` structures once the initial capacity is exhausted.

Each song in the playlist has a **title** and a **rating** (where 1 is the lowest rating and 5 is the highest). The program starts by asking the user for the initial playlist capacity; this number defines the sizes of the following two arrays:

- `songTitles[]` stores the song names.
- `ratings[]` stores the song ratings.

When the arrays are full and the user tries to add another song, the program prints a message stating that the array is full. At that point, it creates two new lists (`ArrayList` for titles and `ArrayList` for ratings), copies all existing songs from the arrays into these lists, and continues using the lists thereafter. From then on, any new songs are added directly to these `ArrayLists`, allowing the playlist to grow dynamically.

The program is menu-driven and repeatedly prompts the user until they choose to exit. All title comparisons should be case-insensitive, song titles must be non-empty and unique, and rating inputs must be validated to be within 1-5.

### Methods to Implement:

- **`addSong(String title, int rating)`**

Adds a new song to the system. If there is space in the arrays, store the song in the arrays; if the arrays are full, switch once to `ArrayLists`, copy all existing songs, and then add the new song. Reject duplicate or empty titles and ratings outside 1-5 with a clear error message.

- **`removeSong(String title)`**

Removes a song by its title. This operation must work whether the system is currently using arrays or `ArrayLists`. If the song does not exist, display "Song not found!".

- **updateSongRating(String title, int newRating)**

Updates the rating of an existing song. First, display the current rating to the user, then request a new rating (1-5). If the user enters the same rating as the current one, reject and re-prompt. If the song is not present, display "Song not found!".

- **searchSong(String title)**

Searches for a song by title and, if found, displays its current rating. If the song does not exist, print "Song not found!". (You are allowed to use built-in string comparison methods for this question).

- **viewAllSongs()**

Displays all songs currently in the playlist with their titles and star ratings. If there are no songs, print "Playlist is empty".

- **exit()**

Terminates the program gracefully by ending the menu loop.

The following table shows sample runs:

|                                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                      |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> Enter initial playlist capacity: 2 === Playlist Manager === 1. Add Song 2. Remove Song 3. Update Song Rating 4. Search Song 5. View All Songs 6. Exit Choose an option: 1 Enter song title: SongA Enter rating (1-5): 5 Song added successfully! === Playlist Manager === Choose an option: 1 Enter song title: SongB </pre> | <pre> === Playlist Manager === 1. Add Song 2. Remove Song 3. Update Song Rating 4. Search Song 5. View All Songs 6. Exit Choose an option: 3 Enter song title to update: SongB Current rating: 3 Enter new rating (1-5): 3 New rating cannot be the same as the current rating. Try again. Enter new rating (1-5): 4 Rating updated. === Playlist Manager === </pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                                                                                                                                                                                                                                                                                                                                                                  |                                                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Enter rating (1-5): 3<br>Song added successfully!<br>=== Playlist Manager ===<br>Choose an option: 1<br>Enter song title: SongC<br>Enter rating (1-5): 4<br>Array full! Switching to dynamic<br>ArrayList...<br>Song added successfully!<br>=== Playlist Manager ===<br>Choose an option: 5<br>Playlist:<br>1. SongA (Rating: 5)<br>2. SongB (Rating: 3)<br>3. SongC (Rating: 4) | Choose an option: 4<br>Enter song title to search: SongD<br>Song not found!<br>=== Playlist Manager ===<br>Choose an option: 6<br>Goodbye! |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|

### 3. Matrix Multiplication

Create a new file under the `lab6` folder named `Lab06_03.java` with a class of the same name.

Your program will perform standard mathematical matrix multiplication. The program should first prompt the user to enter the dimensions (rows and columns) for Matrix A, followed by its integer elements. Then, it should prompt the user for the dimensions and elements of Matrix B.

#### Requirements:

- You must implement a static method: `int[][] multiplyMatrices(int[][] matA, int[][] matB)`
- Before attempting to multiply, your `main` method must check if the two matrices are compatible for multiplication (the number of columns in Matrix A must equal the number of rows in Matrix B).
- If they are not compatible, print an error message and terminate the program.
- If they are compatible, pass them to your method. The method should create a new 2D array for the result, compute the product using nested loops, and return the resulting matrix.
- Print the final result matrix in a neatly aligned grid format.

**Note:** Remember that the resulting matrix will have the number of rows of Matrix A and the number of columns of Matrix B. You may assume all inputs will be valid integers.

The following table shows sample runs:

|                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Example 1 (Valid Dimensions):</b></p> <p>Enter rows for Matrix A: 2</p> <p>Enter columns for Matrix A: 3</p> <p>Enter elements for Matrix A:</p> <p>1 2 3</p> <p>4 5 6</p> <p>Enter rows for Matrix B: 3</p> <p>Enter columns for Matrix B: 2</p> <p>Enter elements for Matrix B:</p> <p>7 8</p> <p>9 10</p> <p>11 12</p> <p>Resulting Matrix:</p> <p>58 64</p> <p>139 154</p> | <p><b>Example 2 (Invalid Dimensions):</b></p> <p>Enter rows for Matrix A: 2</p> <p>Enter columns for Matrix A: 2</p> <p>Enter elements for Matrix A:</p> <p>1 2</p> <p>3 4</p> <p>Enter rows for Matrix B: 3</p> <p>Enter columns for Matrix B: 2</p> <p>Enter elements for Matrix B:</p> <p>1 2</p> <p>3 4</p> <p>5 6</p> <p>Error: Dimensions do not match for multiplication.</p> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|